

- S02.01 MongoDB: Deep Dive
  - SQL vs NoSQL
  - The workout example - flexible data
    - The SQL part
    - The NoSQL part: document
  - Data polymorphism
  - ACID:
  - **Is MongoDB less reliable for ACID than PostgreSQL?**
  - 📌 **Example: Multi-document transaction consistency problem**
    - **Banking database case: Money transfer between two accounts**
    - **PostgreSQL (Robust ACID)**
    - **MongoDB (Risk with multi-document transactions)**
  - 🚨 **Potential problem with MongoDB:**
  - 🔍 **Conclusion**
  - OLTP vs OLAP
    - OLTP
    - OLAP
  - [EXERCISE] questioning an LLM
  - Cases better suited to MongoDB
  - Moral of the story
  - Let's recap
  - NoSQL database performance
  - Conclusion on SQL vs NoSQL - PostgreSQL vs MongoDB
  - Performance
  - PostgreSQL vs MongoDB performance comparison
  - Explanation of query planning in MongoDB
  - Query planning comparison with PostgreSQL

---

## S02.01 MongoDB: Deep Dive

What truly characterizes MongoDB:

- Choosing MongoDB over SQL
- Schema design patterns

- MongoDB-specific features
- Query optimization
- Index creation

Choosing a data store like MongoDB vs a more standard SQL database, is driven by the data and the application that will consume that data.

---

## SQL vs NoSQL

This is the first question you'll probably ask when starting to design your application.

We talked about this last time. But it's interesting to examine this in more detail.

In short, document-based databases make sense for:

- web scale
- flexible data schema

By the way, it's well known that MongoDB is webscale: <https://www.youtube.com/watch?v=b2F-DlItXtZs>

You've moved beyond simple sqlite database or JSON, csv file storage.

What should you turn to next?

Extract:

Some questions when I compare these different backends:

- How often will I read and write the data?
- How often will the data structure change?
- How much of the data is unknown?
- What does the final product look like?

An excellent article on when to use NoSQL: <https://medium.com/@sqlinsix/when-to-use-sql-or-nosql-b50d4a52c157> (available as PDF in the github repo in ./reading)

**Sidenote** NoSQL supports SQL

The article starts with:

NoSQL means not only SQL. With some NoSQL APIs, you may be able to write SQL against them.

And indeed, some NoSQL databases have evolved to support SQL-like query languages or SQL interfaces, even though they weren't originally designed with SQL in mind.

[Atlas SQL Interface](#), [Mongo BI connector](#)

## The workout example - flexible data

Imagine a gym/workout application where you track your sessions.

### The SQL part

There's a series of *entities* compatible with a SQL database:

- a `user` has multiple `sessions`,
- a `gym` has multiple `users`,
- a `user` has a `subscription`,
- `gyms` have multiple `subscriptions` and vice versa.

So subscription management fits well with a relational/SQL database.

The data evolves little and the transaction dimension is important.

### The NoSQL part: document

But there's also a **workout** entity that, by nature, can vary a lot.

- weights and repetitions: dead lifts, clean and jerks, ...
- without weights: dead lifts, clean and jerks, ...
- running, rowing, swimming, ...
- cycling, walking, parachute jumping, ...
- power yoga, pilates, Zumba, ...

A fixed data schema is not suitable.

Another extract from the same article:

*"The complexity of joins would be enormous compared to a person's training, as there can be dozens of combinations. Additionally, workouts such as drop sets where the weight constantly changes and where a maximum (or minimum) repetition range must be respected would be extremely difficult to track with a fixed schema for a SQL database."*

An application exploiting workout data is a good example where a **flexible schema** data structure really makes sense.

You need to think about how the application consumes the data.

For example, if the application retrieves an entire workout for a given day.

In a normalized SQL database, one piece of information exists in a single table.

In such a normalized SQL database, we would need to perform a significant number of joins between the `session` table and those of the different types of `exercises` and the day's workout.

With a document-type NoSQL database (MongoDB), since the workout entity is stored in its entirety as a single piece of information, in a single document, there's no longer a need for these joins

between multiple tables.

**Note:** PostgreSQL now has [JSON data types](#) that can be used for such polymorphic data cases.

## Data polymorphism

When data has fields that vary significantly, a NoSQL document database is better suited for the final application.

On the other hand, stable data that evolves little will be better suited to a classic SQL database.

---

## ACID:

ACID is a set of properties guaranteeing the reliability of transactions in a database:

1. **Atomicity:** A transaction is either fully executed or canceled in case of failure (all or nothing).
2. **Consistency:** A transaction brings the database from one valid state to another valid state, respecting integrity constraints.
3. **Isolation:** Concurrent transactions execute without interfering with each other, as if they were executed sequentially.
4. **Durability:** Once validated, a transaction is permanently recorded, even in case of failure.

These properties are essential for ensuring data integrity in relational databases and some NoSQL databases. 🚀

| Criteria                    | PostgreSQL           | MongoDB                           |
|-----------------------------|----------------------|-----------------------------------|
| Native ACID?                | ✅ Yes (complete)     | ⚠️ Partial (multi-docs expensive) |
| Multi-document transactions | ✅ Yes and optimized  | ⚠️ Yes but expensive              |
| Strong isolation            | ✅ Yes (SERIALIZABLE) | ⚠️ Not as strict                  |
| Scalability                 | ⚠️ Less flexible     | ✅ More flexible for big data      |

- 🚀 PostgreSQL is more reliable for critical transactions, particularly banking or financial.
- ⚡ MongoDB is useful for scalable databases, but ACID is more expensive and less robust than PostgreSQL.

## Is MongoDB less reliable for ACID than PostgreSQL?

👉 **Yes, MongoDB is less strict than PostgreSQL regarding ACID**, particularly for multi-document transactions and concurrent isolation.

## 📌 Example: Multi-document transaction consistency problem

**Banking database case: Money transfer between two accounts**

Imagine you have a database with two collections: - `accounts` stores bank accounts with `balance`  
- `transactions` stores transfer history

### PostgreSQL (Robust ACID)

```
BEGIN;  
  
UPDATE accounts  
SET balance = balance - 500  
WHERE account_id = 1;  
  
UPDATE accounts  
SET balance = balance + 500  
WHERE account_id = 2;  
  
INSERT INTO transactions (from_account, to_account, amount)  
VALUES (1, 2, 500);  
  
COMMIT;
```

SQL

✅ If one of the operations fails, everything is canceled (automatic `ROLLBACK`). ✅ Other concurrent transactions won't see any intermediate state.

---

### MongoDB (Risk with multi-document transactions)

```
const session = db.getMongo().startSession();
session.startTransaction();

try {
  // Debit 500€ from account 1
  session.getDatabase("bank").accounts.updateOne(
    { account_id: 1 },
    { $inc: { balance: -500 } }
  );

  // Credit 500€ to account 2
  session.getDatabase("bank").accounts.updateOne(
    { account_id: 2 },
    { $inc: { balance: 500 } }
  );

  // Add a transaction record
  session.getDatabase("bank").transactions.insertOne(
    { from: 1, to: 2, amount: 500, timestamp: new Date() }
  );

  session.commitTransaction();
} catch (error) {
  session.abortTransaction();
}
session.endSession();
```

### Potential problem with MongoDB:

- Before **MongoDB 4.0**, each operation was independent, so if one update failed, the other could be applied, creating **an inconsistent state**.
- **Multi-document transactions exist**, but are **more expensive in performance and less optimized than PostgreSQL**.
- If the server crashes after updating one document but before the other, **the database can be corrupted**.
- **MongoDB doesn't have advanced isolation levels** like **SERIALIZABLE** in PostgreSQL.

### Conclusion

| Criteria                    | PostgreSQL           | MongoDB                           |
|-----------------------------|----------------------|-----------------------------------|
| Native ACID?                | ✅ Yes (complete)     | ⚠️ Partial (multi-docs expensive) |
| Multi-document transactions | ✅ Yes and optimized  | ⚠️ Yes but expensive              |
| Strong isolation            | ✅ Yes (SERIALIZABLE) | ⚠️ Not as strict                  |
| Scalability                 | ⚠️ Less flexible     | ✅ More flexible for big data      |

🚀 **PostgreSQL** is **more reliable for critical transactions**, particularly **banking or financial**. ⚡ **MongoDB** is useful for **scalable** databases, but **ACID is more expensive and less robust** than PostgreSQL.

## OLTP vs OLAP

Do you need an OLAP or OLTP database?

And in which case will a MongoDB-type database be more suitable than a SQL database?

We consider 2 main types of database usage: transactions or dashboards

### OLTP

On one side, **OLTP** (Online Transaction Processing) systems focus on managing a large number of short and **atomic** transactions in real time. They are optimized for write operations and transactional integrity, commonly used for applications such as e-commerce, banking, and inventory management.

In OLTP systems, we maximize write reliability, data integrity, and database normalization.

Think OLTP == transactions.

### OLAP

On the other side, **OLAP** (Online Analytical Processing) systems are designed to query and analyze large volumes of data, often involving complex joins, aggregations, and data transformations.

These systems are *read-intensive* and require optimized data retrieval. Denormalization simplifies queries and improves performance.

Think OLAP ~= dashboards.

So, the big question is: *should we use NoSQL or SQL for OLAP? for OLTP?*

- For OLTP applications: MongoDB's document side is perfect for managing many small and fast transactions, as each document contains all the data.
- For OLAP applications: MongoDB has certain limitations that make it less ideal for complex analytical queries:

- The aggregation pipeline, used for complex queries, is powerful, but it can become quite complex for analytical queries, which are inherently complicated. A fixed and normalized SQL schema might be simpler.
- In MongoDB, joining large amounts of data belonging to different collections will be memory-intensive and slower compared to SQL databases.
- MongoDB's storage engine is optimized for random access patterns (typical of OLTP) when adequate indexes have been created beforehand, rather than for sequential analyses common in analytical workloads.

MongoDB, as a document-oriented NoSQL database, is optimized for fast read and write operations on **individual documents**, which is ideal for OLTP applications.

However:

For OLTP applications in the strict sense of the term, we would choose a SQL database because it's more ACID than MongoDB.

So

OLTP: Focuses on managing real-time transactional data, emphasizing speed and accuracy for operations like order processing, inventory management, and financial transactions.

OLAP: Centers on complex data analysis and reporting, handling large volumes of historical data to support decision-making processes.

---

## [EXERCISE] questioning an LLM

**Scenario:** if you transfer the join and aggregation workload to the collection level (through regular background tasks for example), does a document database like MongoDB become competitive for OLAP applications?

- Ask an LLM (chatGPT, DeepSeek, Claude.ai, Qwen, Mistral, ) what it thinks.
- Compare the responses 2 by 2
- what are the arguments put forward?
- ask the LLM to argue for this solution

---

## Cases better suited to MongoDB

beyond OLTP / OLAP

MongoDB is particularly well-suited for applications that require flexibility, scalability, and the ability to handle diverse and evolving data types.

- Content Management Systems (CMS) MongoDB's flexible schema allows for the storage and retrieval of various content types, making it ideal for CMS platforms that manage diverse media and documents.



- Internet of Things (IoT) With its ability to handle large volumes of unstructured data, MongoDB is well-suited for IoT applications that collect and process data from numerous devices.
- Mobile Applications Its flexible data model and support for offline data synchronization make MongoDB a strong choice for mobile applications that require seamless data handling across devices.
- E-commerce Platforms MongoDB's scalability and ability to handle diverse product catalogs and customer data make it suitable for e-commerce platforms that need to manage large inventories and user interactions.

## Moral of the story

This phrase highlights when to use NoSQL

extract from: <https://softwareengineering.stackexchange.com/a/355964/440337>

*NOSQL is very much designed as a persistence layer for an application So access is optimized for a single object with children.*

Many things to unpack here:

- **persistence layer** refers to storing and retrieving data so that it survives beyond the application's execution time.

NoSQL (Document) databases are built specifically to meet the **data storage** needs of applications. The important word being "**Application**".

In comparison, SQL databases are designed for data organization and relationships, but application needs being secondary.

- A **single object** refers to a main data entity (like a user profile, a product, a playlist, or a workout)
- The **children** refer to associated data that belongs to or is closely associated with this main object (like user addresses, product reviews, or workout exercises)

Why is this optimized for a *single object* with *children*?

**single** is an important word here:

NoSQL databases physically store the main object and its sub-dependencies together in a **single location** on disk. Instead of splitting associated data across multiple tables like in relational databases.

Retrieving all associated data requires fewer disk operations and is less resource-intensive and therefore faster.

This also implies a key architectural principle: NoSQL databases are designed around the specific ways applications need to access data, rather than maintaining formal data relationships.

Application needs >>> Structured data organization

This makes them particularly efficient when an application needs to quickly retrieve/update an entire object with all its associated data in a single operation.

---

## Let's recap

NoSQL / document when

- evolving data with a structure that can change: flexible schema
- data and associated sub-data are consumed all at once by the application

SQL when - classic OLTP or OLAP - data with a fixed schema - ACID compliance

---

## NoSQL database performance

Several technical factors can affect performance:

- **Indexing:** First, NoSQL databases don't maintain the same type of sophisticated indexes as relational databases for joining data spread across different collections. The role of indexes in MongoDB is to **help locate** individual documents rather than optimize complex queries across multiple documents.
- **Disk access:** Next, when you need to access multiple objects that are not physically stored close to each other, the database must perform multiple random disk searches.

These random searches are much slower than sequential reads, because the disk read head must physically move to different locations. This is why operations that require scanning many documents (like analytical queries) can be slower in MongoDB than in relational databases.

- **Memory management:** Finally, MongoDB's memory management is optimized around the concept of working sets: the documents that are most frequently accessed. On large data volumes that exceed RAM size, MongoDB must constantly swap documents in and out of memory with a probable impact on performance.

On point 2, Disk access: The problem of data scattered around the physical disk at random locations is true for all databases.

But PostgreSQL (for example) is much more optimized for this kind of task than MongoDB is.

There are key architectural differences in how the two databases handle multi-record operations:

- Storage layout: better storage organization and more constraints in PostgreSQL
  - PostgreSQL stores data in tables with fixed schemas, using a concept called "heap files" where records are stored in blocks/pages. These pages are designed for efficient sequential scanning.
  - MongoDB stores each document independently, potentially with variable sizes and schemas. This can lead to more fragmentation and less efficient sequential access.

- Data access methods: better query planning
  - PostgreSQL has very sophisticated and therefore very efficient query planning that chooses between several access methods:
  - Sequential scans that read pages efficiently in order
  - Index scans that can use multiple indexes simultaneously
  - Bitmap scans that can combine results from multiple index types
  - MongoDB primarily relies on single-index scans or collection scans, with less ability to combine access methods
- Join operations:
  - PostgreSQL has specialized algorithms (hash joins, merge joins, nested loops) that are optimized to efficiently combine data from multiple tables
  - MongoDB must perform document-by-document searches when combining data between collections, which often means more random I/O
- Buffer management:
  - PostgreSQL's buffer manager is designed to optimize sequential and random access patterns
  - MongoDB's WiredTiger storage engine is primarily optimized for random access patterns

So, while both databases must perform disk operations, PostgreSQL includes specific optimizations to efficiently handle multi-record operations, particularly when it comes to joins and large table scans.

These optimizations are less present in MongoDB because its architecture prioritizes single-document operations.

## **Conclusion on SQL vs NoSQL - PostgreSQL vs MongoDB**

### **Impossible to escape complexity.**

Speed is not the only factor to consider when choosing a database.

Regardless of any technical consideration, engineer availability and productivity have more impact on costs than a few milliseconds gained on a query.

What motivates MongoDB adoption over PostgreSQL is

- How the application consumes data
- storage and retrieval of single records
- data complexity

---

## **Performance**

### **PostgreSQL vs MongoDB performance comparison**

Read this article <https://medium.com/@vosarat1995/postgres-vs-mongo-performance-comparison-for-semi-structured-data-9a5ec6486cf6>

The author creates a large dataset and tests the performance of MongoDB and PostgreSQL.

| Test                             | Mongo     | PostgreSQL |
|----------------------------------|-----------|------------|
| Index creation on empty database | 152.9 ms  | 402.0 ms   |
| Batch insertion                  | 53.50 ms  | 219.15 ms  |
| One-by-one insertion             | 319.3 ms  | 641.9 ms   |
| Multiple record reading          | 21.83 ms  | 66.63 ms   |
| Index creation on 1000 rows      | 1.563 s   | 1.916 s    |
| Complex Query                    | 174.51 ms | 60.43 ms   |

MongoDB wins most of the time.

But PostgreSQL wins in the final sprint!

Furthermore: *"It is worth noting that the tests were on semi-structured data which is the realm of Mongo."*

## Explanation of query planning in MongoDB

Without going into the details of index creation with MongoDB for now, let's analyze query planning.

To `EXPLAIN` a query:

- direct query: `db.movies.find(<query predicate>).explain("executionStats")`
- aggregation pipelines:  
`db.movies.explain("executionStats").aggregate(<the_pipeline>)`

Note the following elements

- `totalDocsExamined`
- `nReturned`
- `executionTimeMillisEstimate`: estimates the cumulative execution time for the pipeline up to and including the stage in question.
- `rejectedPlans`
- `stage`: COLLSCAN, IXSCAN

compare: `db.movies.find({ "title": "The Perils of Pauline" }).explain("executionStats")`

Then, create an index `db.movies.createIndex({ title: 1 })`

again `db.movies.find({ "title": "The Perils of Pauline" }).explain("executionStats")`

Drop the index `db.movies.dropIndex({title : 1})`

Explain an aggregation pipeline `db.movies.explain("executionStats").aggregate([ { $group: { _id: "$genres", averageRating: { $avg: "$imdb.rating" } } } ] )`

## Query planning comparison with PostgreSQL

Consider a similar query scenario that involves joining/linking data and aggregating results.

**Query:** *find all movies from the 1990s with their directors, grouped by director with average ratings.*

First, MongoDB:

```
db.movies.aggregate([
  { $match: {
    year: { $gte: 1990, $lt: 2000 }
  } },
  { $unwind: "$directors" },
  { $group: {
    _id: "$directors",
    avgRating: { $avg: "$imdb.rating" },
    movieCount: { $sum: 1 }
  } }
]).explain("executionStats")
```

Bash

The plan is:

```
{
  "stages": [
    {
      "$cursor": {
        "queryPlanner": {
          "plannerVersion": 1,
          "namespace": "sample_mflix.movies",
          "indexFilterSet": false,
          "parsedQuery": {
            "year": { "$gte": 1990, "$lt": 2000 }
          },
          "winningPlan": {
            "stage": "COLLSCAN", // Full collection scan
            "filter": { "year": { "$gte": 1990, "$lt": 2000 } }
          },
          "rejectedPlans": []
        }
      }
    },
    { "$unwind": "$directors" }, // Memory-intensive operation
    {
      "$group": {
        "_id": "$directors",
        "avgRating": { "$avg": "$imdb.rating" },
        "movieCount": { "$sum": 1 }
      }
    }
  ]
}
```

Now, the PostgreSQL equivalent (assuming a normalized schema):

```
EXPLAIN ANALYZE
SELECT d.name as director,
       AVG(m.rating) as avg_rating,
       COUNT(*) as movie_count
FROM movies m
JOIN movie_directors md ON m.id = md.movie_id
JOIN directors d ON md.director_id = d.id
WHERE m.year >= 1990 AND m.year < 2000
GROUP BY d.name;
```

Which gives:

# QUERY PLAN

```
-----
HashAggregate (cost=245.97..247.97 rows=200)
  Group Key: d.name
  -> Hash Join (cost=121.67..237.42 rows=1710)
    Hash Cond: (md.director_id = d.id)
    -> Hash Join (cost=66.50..164.42 rows=1710)
      Hash Cond: (md.movie_id = m.id)
      -> Seq Scan on movie_directors md
      -> Hash (cost=58.00..58.00 rows=680)
        -> Index Scan using movies_year_idx on movies m
            Index Cond: (year >= 1990 AND year < 2000)
    -> Hash (cost=40.50..40.50 rows=1173)
      -> Seq Scan on directors d
```

The main differences in query planning are:

## 1. Join handling:

- PostgreSQL uses **hash joins** to efficiently combine data from multiple tables
- MongoDB must `$unwind` the nested director array, which creates multiple documents in memory

## 2. Index usage:

- PostgreSQL can use **multiple indexes** simultaneously and choose different join strategies
- MongoDB generally relies on a single index per aggregation stage

## 3. Memory management:

- PostgreSQL's hash joins create hash tables in memory with disk spillover if necessary
- MongoDB's `$unwind` and `$group` stages must keep their data in memory

## 4. Operation order:

- PostgreSQL's query planner can **reorder operations** for efficiency
- MongoDB must process aggregation pipeline stages in order

## 5. Statistics usage:

- PostgreSQL's explanation shows detailed cost estimates based on table statistics
- MongoDB's explanation focuses more on operation type and index usage

The PostgreSQL plan shows that it can:

- Use an index scan for the year filter
- Create hash tables for efficient joining
- Perform grouping with hash aggregation

- Estimate costs and row counts at each stage

👁️👁️👁️ This structured approach to complex queries is why PostgreSQL is often more performant for analytical workloads involving multiple tables/collections and aggregations.